JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class BufferedWriter

java.lang.Object
    java.io.Writer
        java.io.BufferedWriter

All Implemented Interfaces:

Closeable, Flushable, Appendable, AutoCloseable

```
public class BufferedWriter
extends Writer
```

Writes text to a character-output stream, buffering characters so as to provide for the efficient writing of single characters, arrays, and strings.

The buffer size may be specified, or the default size may be accepted. The default is large enough for most purposes.

A `newLine()` method is provided, which uses the platform's own notion of line separator as defined by the system property line.separator. Not all platforms use the newline character ('\n') to terminate lines. Calling this method to terminate each output line is therefore preferred to writing a newline character directly.

In general, a `Writer` sends its output immediately to the underlying character or byte stream. Unless prompt output is required, it is advisable to wrap a `BufferedWriter` around any `Writer` whose `write()` operations may be costly, such as `FileWriters` and `OutputStreamWriters`. For example,

```
    PrintWriter out = new PrintWriter(new BufferedWriter(new FileWriter("foo.out"
```

will buffer the `PrintWriter`'s output to the file. Without buffering, each invocation of a `print()` method would cause characters to be converted into bytes that would then be written immediately to the file, which can be very inefficient.

API Note:

Once wrapped in a `BufferedWriter`, the underlying `Writer` should not be used directly nor wrapped with another writer.

Since:

1.1

See Also:

PrintWriter,
FileWriter,
OutputStreamWriter,

Files.newBufferedWriter(Path, Charset, OpenOption...)

Field Summary

Fields declared in class Writer
lock

Constructor Summary

Constructors

Constructor	Description
BufferedWriter(Writer out)	Creates a buffered character-output stream that uses a default-sized output buffer.
BufferedWriter(Writer out, int sz)	Creates a new buffered character-output stream that uses an output buffer of the given size.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	close()	Closes the stream, flushing it first.
void	flush()	Flushes the stream.
void	newLine()	Writes a line separator.
void	write(char[] cbuf, int off, int len)	Writes a portion of an array of characters.
void	write(int c)	Writes a single character.
void	write(String s, int off, int len)	Writes a portion of a String.

Methods declared in class Writer

`append`, `append`, `append`, `nullWriter`, `write`, `write`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

BufferedWriter

```
public BufferedWriter(Writer out)
```

Creates a buffered character-output stream that uses a default-sized output buffer.

Parameters:

`out` - A Writer

BufferedWriter

```
public BufferedWriter(Writer out,
                      int sz)
```

Creates a new buffered character-output stream that uses an output buffer of the given size.

Parameters:

`out` - A Writer

`sz` - Output-buffer size, a positive integer

Throws:

`IllegalArgumentException` - If `sz <= 0`

Method Details

write

```
public void write(int c)
           throws IOException
```

Writes a single character.

Overrides:

`write` in class `Writer`

Parameters:

`c` - int specifying a character to be written

Throws:

`IOException` - If an I/O error occurs

write

```
public void write(char[] cbuf,
                  int off,
                  int len)
           throws IOException
```

Writes a portion of an array of characters.

Ordinarily this method stores characters from the given array into this stream's buffer, flushing the buffer to the underlying stream as needed. If the requested length is at least as large as the buffer, however, then this method will flush the buffer and write the characters directly to the underlying stream. Thus redundant `BufferedWriters` will not copy data unnecessarily.

Specified by:

`write` in class `Writer`

Parameters:

`cbuf` - A character array

`off` - Offset from which to start reading characters

`len` - Number of characters to write

Throws:

`IndexOutOfBoundsException` - If `off` is negative, or `len` is negative, or `off + len` is negative or greater than the length of the given array

`IOException` - If an I/O error occurs

write

```
public void write(String s,
                  int off,
                  int len)
           throws IOException
```

Writes a portion of a String.

Overrides:

`write` in class `Writer`

Implementation Requirements:

While the specification of this method in the superclass recommends that an IndexOutOfBoundsException be thrown if len is negative or off + len is negative, the implementation in this class does not throw such an exception in these cases but instead simply writes no characters.

Parameters:

s - String to be written

off - Offset from which to start reading characters

len - Number of characters to be written

Throws:

IndexOutOfBoundsException - If off is negative, or off + len is greater than the length of the given string

IOException - If an I/O error occurs

newLine

```
public void newLine()
             throws IOException
```

Writes a line separator. The line separator string is defined by the system property line.separator, and is not necessarily a single newline ('\n') character.

Throws:

IOException - If an I/O error occurs

flush

```
public void flush()
           throws IOException
```

Flushes the stream.

Specified by:

flush in interface Flushable

Specified by:

flush in class Writer

Throws:

IOException - If an I/O error occurs

close

```
public void close()
           throws IOException
```

Description copied from class: `Writer`

Closes the stream, flushing it first. Once the stream has been closed, further write() or flush() invocations will cause an IOException to be thrown. Closing a previously closed stream has no effect.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in class `Writer`

Throws:

`IOException` - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.